

CS2306 Lab2: 4-bit Adder 设计与验证

姓名： 吴恒涛 学号： 524031910038

摘要

本报告详述了基于 Xilinx Vivado 开发平台的 4-bit 全加器硬件系统的开发全流程。实验从 1-bit 全加器的底层逻辑门抽象入手，利用结构化建模实现 4-bit 级联加法器。随后，通过 Testbench 进行了功能级仿真验证，并最终编写管脚约束文件以生成 Bitstream 配置文件。工程中还结合了给定的 display IP 核网表文件，完成了外部拨码开关输入与 LED、七段数码管协同结果显示的物理层映射。

目录

1	实验导言与系统概述	2
2	硬件架构与逻辑实现	2
2.1	基础门级加法器设计	2
2.2	级联 4-bit 加法器	2
2.3	顶层模块与外设驱动	3
3	仿真测试平台构造	3
4	物理综合与实体验证	3
4.1	引脚映射与电平约束	3
4.2	上板效果与现象分析	4
5	工程反思与总结	4
A	附录：实验源代码汇总	5
A.1	1-bit 全加器 (adder_1bit.v)	5
A.2	4-bit 全加器 (adder_4bits.v)	5
A.3	顶层控制模块 (Top.v)	6
A.4	功能仿真文件 (adder_4bits_tb.v)	7
A.5	引脚约束文件 (lab02_xdc.xdc)	8

1 实验导言与系统概述

本次 FPGA 基础实验的核心目的是掌握 Verilog HDL 进行简单逻辑设计的基本规范，并熟悉 Vivado 环境下的完整工作流。4-bit Adder（四位加法器）作为一个经典的组合逻辑电路，完整涵盖了数字系统设计的几大核心要素：底层逻辑门搭建、模块例化、功能仿真与 I/O 驱动绑定。

整个工程按照功能划分为多个层级：最底层的 1 位全加器 `adder_1bit`；中间层的 4 位全加器 `adder_4bits`；用于功能验证的激励仿真文件 `adder_4bits_tb`；以及负责整合时钟、显示驱动 IP 核与加法器本体的顶层模块 `Top`。最后通过 `lab02_xdc` 约束文件将逻辑网络映射到实际的物理引脚。

2 硬件架构与逻辑实现

2.1 基础门级加法器设计

系统的最底层组件为 1-bit 全加器 `adder_1bit`，该模块直接调用了 Verilog 的内建逻辑门原语（primitive）。利用 `and`、`or` 与 `xor` 门，根据全加器的布尔代数表达式严格构建了进位信号 `c1`、`c2`、`c3` 与中间求和信号 `s1` 的拓扑关系。这种门级描述方式直观展现了数字电路的硬件底层结构。

Listing 1: 1-bit 全加器核心逻辑

```
1 wire s1,c1,c2,c3;
2 and (c1,a,b),
3     (c2,b,ci),
4     (c3,a,ci);
5 xor (s1,a,b),
6     (s,s1,ci);
7 or (co,c1,c2,c3);
```

2.2 级联 4-bit 加法器

在完成了单比特加法器后，通过结构化建模的方式，例化了 4 个 `adder_1bit` 模块构成 `adder_4bits`。设计中引入了内部网络 `wire [2:0] ct;`，用于承接上一位加法器的进位输出并将其作为下一位加法器的进位输入（Ripple Carry Adder 结构）。

2.3 顶层模块与外设驱动

由于实验板的系统时钟为高频差分信号 (clk_p 与 clk_n)，设计中使用了 Xilinx 的全局时钟缓冲原语 IBUFGDS 将其转换为单端时钟信号 CLK_i。为了便于结果观测，顶层模块将加法器的 4 位本位和 s 与 1 位进位输出 co 拼接，形成 5 位宽度的 sum 信号。最终，将该综合结果与外部的 display 网表 IP 核连接，以驱动 LED 与七段数码管。

3 仿真测试平台构造

单纯的模块编写无法保证逻辑绝对正确，构建完备的测试平台 (Testbench) 是硬件工程不可或缺的一环。在 adder_4bits_tb 中，我对被测的 4 位加法器模块进行了实例化，并提供了一系列时间间隔为 100ns 的激励向量。

测试用例覆盖了无进位与有进位的多种场景，例如：a=0001, b=0010; a=0010, b=0100; 以及包含进位输入 ci=1 的满载测试 a=1111, b=0001。

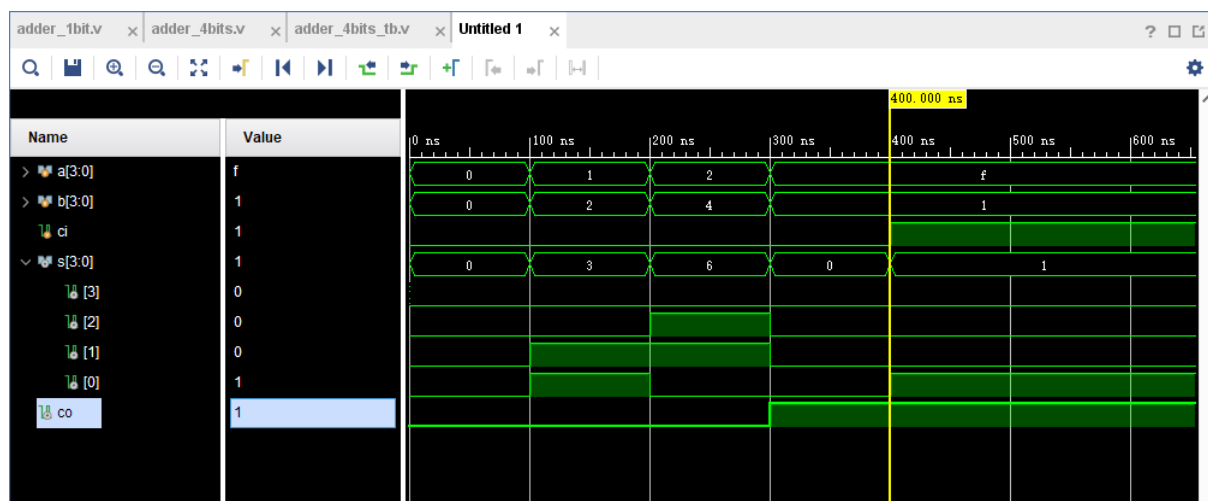


图 1: 4 位加法器功能仿真波形图

从仿真波形可以清晰地观察到，输出总线 s 与进位标志 co 在每个激励时刻都给出了精确的算术和结果，仿真结果与预期逻辑功能高度一致，证明核心算术电路设计无误。

4 物理综合与实体验证

4.1 引脚映射与电平约束

在 XDC 约束文件中，我严格根据实验板的硬件手册配置了各端口的物理管脚与电气标准。差分时钟引脚采用 LVDS 标准；用于输入加数 a 和 b 的 8 个拨码开关配置为 LVCMOS15 低压标准；而用于显示的 LED 及数码管端口则统一绑定至 LVCMOS33 标准。

4.2 上板效果与现象分析

将生成的 bit 文件下载至 FPGA 芯片后，系统表现出与仿真完全吻合的物理特性。

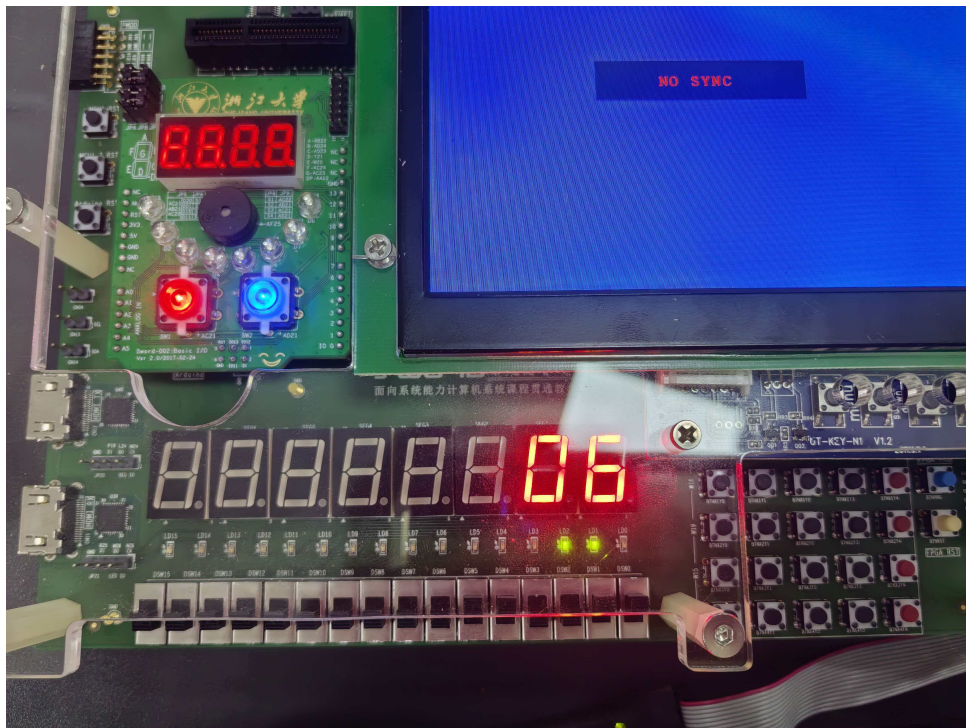


图 2: 实验板上板运行实拍图

通过拨动板载的 8 个 Switch（低 4 位与次高 4 位分别代表两个加数），实验板的 LED 阵列与七段数码管作出了实时响应：LED 的低 4 位直观呈现了二进制形式的本位和，第 5 位 LED 正确指示了最高位进位；同时，七段数码管也准确显示出了当前运算结果的十进制/十六进制数值。软硬件的完美对应验证了本工程从代码设计到物理映射的可靠性。

5 工程反思与总结

通过本次 4-bit Adder 的实验，我首次深入体验了基于 Verilog 结构化建模构建数字系统的全过程。与高级编程语言的软件设计不同，硬件描述语言的组件例化更像是在“接线图”上进行模块化组装，将 1-bit 加法器的进位链式连接为 4-bit 加法器的过程，让我直观感受到了底层电路的并行与级联特性。

此外，引入 display 网表 IP 核的步骤也极大地拓宽了我的工程视野。这让我意识到，在现代 FPGA 体系结构设计中，合理调用与封装成熟的第三方或系统 IP 核是加速开发周期、降低设计复杂度的关键手段。在后续的计算机系统结构课程中，我将尝试把这些模块化的硬件思维应用到更复杂的运算器或存储系统的设计中。

A 附录：实验源代码汇总

A.1 1-bit 全加器 (adder_1bit.v)

```
1 `timescale 1ns / 1ps
2 // Module Name: adder_1bit
3 module adder_1bit(
4     input a,
5     input b,
6     input ci,
7     input s,
8     output co
9 );
10 wire s1,c1,c2,c3;
11
12 and (c1,a,b),
13     (c2,b,ci),
14     (c3,a,ci);
15
16 xor (s1,a,b),
17     (s,s1,ci);
18
19 or (co,c1,c2,c3);
20 endmodule
```

A.2 4-bit 全加器 (adder_4bits.v)

```
1 `timescale 1ns / 1ps
2
3 // Module Name: adder_4bits
4 module adder_4bits(
5     input [3:0] a,
6     input [3:0] b,
7     input ci,
8     input [3:0] s,
9     output co
10 );
11 wire [2:0] ct;
```

```

12
13     adder_1bit a1(.a(a[0]),.b(b[0]),.ci(ci),.s(s[0]),.co(ct[0])),
14                 a2(.a(a[1]),.b(b[1]),.ci(ct[0]),.s(s[1]),.co(ct[1])),
15                 a3(.a(a[2]),.b(b[2]),.ci(ct[1]),.s(s[2]),.co(ct[2])),
16                 a4(.a(a[3]),.b(b[3]),.ci(ct[2]),.s(s[3]),.co(co));
17
18 endmodule

```

A.3 顶层控制模块 (Top.v)

```

1  `timescale 1ns / 1ps
2
3  // Module Name: Top
4
5  module Top(
6      input clk_p,
7      input clk_n,
8      input [3:0] a,
9      input [3:0] b,
10     input reset,
11
12     output led_clk,
13     output led_do,
14     output led_en,
15
16     output wire seg_clk,
17     output wire seg_en,
18     output wire seg_do
19 );
20
21 wire CLK_i;
22 wire Clk_25M;
23
24 IBUFGDS IBUFGDS_inst (
25     .O(CLK_i),
26     .I(clk_p),
27     .IB(clk_n)
28 );

```

```

29
30     wire [3:0] s;
31     wire co;
32     wire [4:0] sum;
33     assign sum = {co, s};
34
35     adder_4bits u1 (
36         .a(a),
37         .b(b),
38         .ci(1'b0),
39         .s(s),
40         .co(co)
41     );
42
43     reg [1:0] clkdiv;
44     always @(posedge CLK_i) clkdiv <= clkdiv + 1;
45     assign Clk_25M = clkdiv[1];
46
47     display DISPLAY (
48         .clk(Clk_25M),
49         .rst(1'b0),
50         .en(8'b11),
51         .data({27'b0, sum}),
52         .dot(8'b0),
53         .led(~{11'b0, sum}),
54         .led_clk(led_clk),
55         .led_en(led_en),
56         .led_do(led_do),
57         .seg_clk(seg_clk),
58         .seg_en(seg_en),
59         .seg_do(seg_do)
60     );
61
62     endmodule

```

A.4 功能仿真文件 (adder_4bits_tb.v)

```

1 `timescale 1ns / 1ps

```

```

2
3 // Module Name: adder_4bits_tb
4
5 module adder_4bits_tb(
6
7     );
8     reg [3:0] a;
9     reg [3:0] b;
10    reg ci;
11    wire [3:0] s;
12    wire co;
13
14    adder_4bits u0(.a(a),.b(b),.ci(ci),.s(s),.co(co));
15
16    initial begin
17        a=0;
18        b=0;
19        ci=0;
20        #100;
21        a=4'b0001;
22        b=4'b0010;
23        #100;
24        a=4'b0010;
25        b=4'b0100;
26        #100;
27        a=4'b1111;
28        b=4'b0001;
29        #100;
30        ci=1'b1;
31    end
32 endmodule

```

A.5 引脚约束文件 (lab02_xdc.xdc)

```

1 set_property PACKAGE_PIN AC18 [get_ports clk_p]
2 set_property IOSTANDARD LVDS [get_ports clk_p]
3
4 set_property PACKAGE_PIN AA12 [get_ports {a[3]}]

```



```

5 set_property PACKAGE_PIN AA13 [get_ports {a[2]}]
6 set_property PACKAGE_PIN AB10 [get_ports {a[1]}]
7 set_property PACKAGE_PIN AA10 [get_ports {a[0]}]
8 set_property IOSTANDARD LVCMOS15 [get_ports {a[3]}]
9 set_property IOSTANDARD LVCMOS15 [get_ports {a[2]}]
10 set_property IOSTANDARD LVCMOS15 [get_ports {a[1]}]
11 set_property IOSTANDARD LVCMOS15 [get_ports {a[0]}]
12
13 set_property PACKAGE_PIN AD10 [get_ports {b[3]}]
14 set_property PACKAGE_PIN AD11 [get_ports {b[2]}]
15 set_property PACKAGE_PIN Y12 [get_ports {b[1]}]
16 set_property PACKAGE_PIN Y13 [get_ports {b[0]}]
17 set_property IOSTANDARD LVCMOS15 [get_ports {b[3]}]
18 set_property IOSTANDARD LVCMOS15 [get_ports {b[2]}]
19 set_property IOSTANDARD LVCMOS15 [get_ports {b[1]}]
20 set_property IOSTANDARD LVCMOS15 [get_ports {b[0]}]
21
22 set_property PACKAGE_PIN N26 [get_ports led_clk]
23 set_property PACKAGE_PIN M26 [get_ports led_do]
24 set_property PACKAGE_PIN P18 [get_ports led_en]
25 set_property IOSTANDARD LVCMOS33 [get_ports led_clk]
26 set_property IOSTANDARD LVCMOS33 [get_ports led_do]
27 set_property IOSTANDARD LVCMOS33 [get_ports led_en]
28
29 set_property PACKAGE_PIN M24 [get_ports seg_clk]
30 set_property PACKAGE_PIN L24 [get_ports seg_do]
31 set_property PACKAGE_PIN R18 [get_ports seg_en]
32 set_property IOSTANDARD LVCMOS33 [get_ports seg_clk]
33 set_property IOSTANDARD LVCMOS33 [get_ports seg_do]
34 set_property IOSTANDARD LVCMOS33 [get_ports seg_en]

```